ระบบนี้เป็น React + Vite ที่ build ออกมาเป็นไฟล์ static (HTML/JS/CSS) ล้วน ๆ ไม่ต้องมี Node หรือฐานข้อมูลบนเครื่องปลายทาง ขอแค่มี Web Server ที่เสิร์ฟไฟล์ และตั้งให้ "ทุกเส้นทางตกกลับมาที่ index.html"

## ขั้นที่ 1 เตรียมของ

- เครื่องสำหรับ build: Node.js 20 ขึ้นไป + npm + Git (ใช้ครั้งเดียวตอน build จะเป็นเครื่องของคุณเองหรือ CI ก็ได้)
- ที่วางเว็บ: Server / VPS / Cloud ใดก็ได้ หรือบริการ Static Hosting (ดูขั้นที่ 3)
- โดเมนหรือซับโดเมน เช่น portal.example.go.th ชี้ DNS (A/CNAME) มาที่เครื่องปลายทาง
- ใบรับรอง HTTPS (ฟรีจาก Let's Encrypt ได้) แนะนำให้ใช้เสมอ และบังคับสำหรับฟีเจอร์ไมโครโฟน (บันทึกเสียงประชุม) กับปุ่มระบุตำแหน่งปัจจุบัน ที่เบราว์เซอร์อนุญาตเฉพาะบน HTTPS

## ขั้นที่ 2 Build เป็นไฟล์ static

```
git clone <URL ของ repo> codi-portal && cd codi-portal
npm ci
npm run build
# ได้โฟลเดอร์ dist/ (ประมาณ 4 MB) = เว็บทั้งหมดที่ต้องเอาไปวาง
```

คำสั่ง npm run build จะตรวจ TypeScript ให้ด้วย ต้องไม่มี error ผลลัพธ์คือโฟลเดอร์ dist/ (ประมาณ 4 MB)

## ขั้นที่ 3 เลือกวิธีวางไฟล์

### Nginx บน Linux Server

เหมาะกับ VPS / เครื่องของหน่วยงาน / Cloud VM (Ubuntu) — วางไฟล์ dist แล้วตั้ง Nginx ให้ทุกเส้นทางตกกลับมาที่ index.html

**คัดลอกไฟล์ขึ้นเครื่อง**

```
scp -r dist/* user@SERVER:/var/www/codi-portal/
```

**/etc/nginx/sites-available/codi-portal**

```
server {
  listen 80;
  server_name portal.example.go.th;
  root /var/www/codi-portal;
  index index.html;

  gzip on;
  gzip_types text/css application/javascript application/json image/svg+xml;

  location /assets/ {
    expires 1y;
    add_header Cache-Control "public, immutable";
  }

  location / {
    try_files $uri $uri/ /index.html;
    add_header Cache-Control "no-cache";
  }
}
```

เปิดใช้งาน

```
sudo ln -s /etc/nginx/sites-available/codi-portal /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

## Docker

ในโปรเจกต์มี Dockerfile และ nginx.conf ให้แล้ว — build image ครั้งเดียว เอาไปรันที่ไหนก็ได้ (Server, Kubernetes, Cloud Run, ECS ฯลฯ)

Build และรัน

```
docker build -t codi-portal .
docker run -d --name codi-portal -p 80:80 --restart unless-stopped codi-portal
```

หรือผ่าน Registry

```
docker tag codi-portal registry.example.com/codi/codi-portal:1.0.0
docker push registry.example.com/codi/codi-portal:1.0.0
```

## Apache

วางไฟล์ใน DocumentRoot แล้วเพิ่ม .htaccess (ต้องเปิด mod_rewrite และ AllowOverride All)

คัดลอกไฟล์

```
scp -r dist/* user@SERVER:/var/www/html/codi-portal/
```

.htaccess (วางข้าง index.html)

```
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /
  RewriteRule ^index\.html$ - [L]
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteCond %{REQUEST_FILENAME} !-d
  RewriteRule . /index.html [L]
</IfModule>
```

## Windows IIS

ติดตั้ง URL Rewrite Module แล้ววางไฟล์ใน Site ของ IIS พร้อม web.config

web.config (วางข้าง index.html)

```
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="SPA" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="/index.html" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

### Static Hosting อื่น

Netlify / Cloudflare Pages / GitHub Pages / AWS S3 + CloudFront / Firebase Hosting — ใช้ได้หมด เพราะเว็บนี้เป็นไฟล์ static (ไม่มี Server ฝั่งหลังบ้าน)

**ตั้งค่าที่เหมือนกันทุกที่**

```
Build command : npm run build
Output folder : dist
Node version  : 20 ขึ้นไป
```

**Netlify / Cloudflare Pages: ไฟล์ public/_redirects**

```
/*    /index.html   200
```

**S3 + CloudFront: ตั้ง Error page**

```
403 และ 404 → /index.html (HTTP 200)
```

GitHub Pages ไม่รองรับ SPA fallback ตรง ๆ ต้องใช้ไฟล์ 404.html ที่เป็นสำเนา index.html

## ขั้นที่ 4 ผูกโดเมนและเปิด HTTPS (ตัวอย่าง Ubuntu + Nginx)

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d portal.example.go.th
# ต่ออายุอัตโนมัติ — ทดสอบด้วย: sudo certbot renew --dry-run
```

## ขั้นที่ 5 ตรวจหลังขึ้นระบบ

- เปิด /login แล้วล็อกอิน demo@codi.or.th / 123456 (บัญชีทดลองในเบราว์เซอร์)
- เปิด URL ลึก ๆ เช่น /tor/sitemap แล้วกดรีเฟรช ต้องไม่ขึ้น 404 (ถ้าขึ้น แปลว่ายังไม่ได้ตั้ง SPA fallback)
- เปิด /community/map ต้องเห็นแผนที่ถนน (ถ้าว่าง ให้ตรวจว่าเครื่องผู้ใช้ออกอินเทอร์เน็ตไปที่ tile.openstreetmap.org ได้)
- เปิดผ่านมือถือ ตรวจว่าเมนูและหน้าหลักแสดงถูกต้อง

## ขั้นที่ 6 อัปเดตเวอร์ชันครั้งต่อไป

```
git pull
npm ci
npm run build
scp -r dist/* user@SERVER:/var/www/codi-portal/   # หรือ docker build + docker run ใหม่
```

ไฟล์ใน assets/ มีชื่อแฮชใหม่ทุกครั้ง ผู้ใช้จึงได้เวอร์ชันล่าสุดทันทีที่รีเฟรช (index.html ตั้ง no-cache ไว้แล้ว) ควรเก็บ dist ของเวอร์ชันก่อนหน้าไว้เผื่อย้อนกลับ

## ข้อควรระวัง

| | |
|---|---|
| **ไม่มี Backend** | ข้อมูลทุกอย่างอยู่ใน localStorage ของเบราว์เซอร์แต่ละเครื่อง ไม่แชร์ข้ามผู้ใช้ และหายเมื่อล้างข้อมูลเบราว์เซอร์ — เหมาะกับ Demo/UAT เท่านั้น |
| **ติดตั้งไว้ใต้โฟลเดอร์ย่อย (เช่น /codi/)** | ต้องตั้ง base: "/codi/" ใน vite.config.ts และ basename ใน BrowserRouter ก่อน build ไม่เช่นนั้นไฟล์และเส้นทางจะหาไม่เจอ ถ้าใช้ทั้งโดเมนหรือซับโดเมน (portal.example.go.th) ไม่ต้องแก้ |
| **แผนที่** | ใช้ภาพแผนที่และการค้นหาตำแหน่งจาก OpenStreetMap (ฟรีแต่มีนโยบายการใช้งาน) ถ้าผู้ใช้เยอะหรือใช้ในเครือข่ายปิด ควรติดตั้ง Tile Server ของตัวเอง |

| | |
|---|---|
| **Node 24 / TypeScript** | โปรเจกต์ใช้ TypeScript 5.9 ห้ามอัปเกรดเป็น 7 (ตัว compile หายบนเครื่องนี้) ให้ใช้ npm ci เพื่อได้เวอร์ชันตาม package-lock.json |
| **หน้าภายในทีม** | หน้า CODI TOR และบัญชี demo จะเปิดให้ทุกคนที่เข้า URL ได้ ถ้าเปิดสาธารณะควรซ่อนหรือกั้นด้วย Basic Auth / VPN |

ยังใช้ Vercel ได้เหมือนเดิม: npx vercel deploy --prod --yes (ในโปรเจกต์มี vercel.json ตั้ง SPA fallback ไว้แล้ว)